Assignment 1

Name: Soham Somnath Kulkarni
Class: BEIT
Roll Number: 407I081

Code and Output:

```python
import re


STOP_WORDS = {
    "the", "is", "in", "and", "to", "of", "a", "an", "that", "this", "on", "for",
    "it", "by", "with", "as", "at", "from", "are", "was", "were", "be", "been"
}


SUFFIXES = ["ing", "edly", "ed", "ly", "es", "s", "ment", "ness", "tion", "able", "al", "ful", "less"]


EQUIVALENT_ENDINGS = {
    "b": "pt",   # absorb → absorpt
    "y": "i",    # happy → happi
    "ive": "ion" # active → action
}


def remove_stop_words(words):
    """Removes common high-frequency words (stopwords)."""
    return [w for w in words if w.lower() not in STOP_WORDS]
```

```python
def strip_suffix(word):
    """Removes the longest valid suffix based on predefined rules."""
    for suffix in sorted(SUFFIXES, key=len, reverse=True):
        if word.endswith(suffix) and len(word) - len(suffix) > 2:
            return word[:-len(suffix)]
    return word
```

```python
def detect_equivalent_stems(stem):
    """Maps stems to equivalent forms using known endings."""
    for a, b in EQUIVALENT_ENDINGS.items():
        if stem.endswith(b):
            return stem[:-len(b)] + a
        elif stem.endswith(a):
            return stem[:-len(a)] + b
    return stem
```

```python
def conflation_algorithm(text):
    # Step 1: Tokenize and remove stop words
    words = re.findall(r'\b[a-zA-Z]+\b', text)
    filtered_words = remove_stop_words(words)

    # Step 2: Apply suffix stripping
    stems = [strip_suffix(word.lower()) for word in filtered_words]

    # Step 3: Detect equivalent stems
    normalized_stems = [detect_equivalent_stems(stem) for stem in stems]
```

```python
def conflation_algorithm(text):
    # Step 1: Tokenize and remove stop words
    words = re.findall(r'\b[a-zA-Z]+\b', text)
    filtered_words = remove_stop_words(words)

    # Step 2: Apply suffix stripping
    stems = [strip_suffix(word.lower()) for word in filtered_words]

    # Step 3: Detect equivalent stems
    normalized_stems = [detect_equivalent_stems(stem) for stem in stems]

    # Step 4: Generate document representative (unique stems)
    representative = sorted(set(normalized_stems))

    return representative
```

```python
if __name__ == "__main__":
    sample_text = """
    Information retrieval systems aim to retrieve relevant documents.
    The retrieval of information involves processing, indexing and ranking.
    """
    result = conflation_algorithm(sample_text)
    print("Document Representative:\n", result)
```

```
Document Representative:
 ['aim', 'document', 'index', 'informa', 'involv', 'process', 'rank', 'relevant', 'retriev', 'retrieve', 'system']
```